

C++ 类和动态内存分配完整自学手册

目录

1. 动态内存分配基础
 2. 特殊成员函数详解
 3. 复制构造函数
 4. 赋值运算符重载
 5. 深度复制与浅度复制
 6. 静态类成员
 7. 运算符重载进阶
 8. 构造函数中使用new的注意事项
 9. 指向对象的指针
 10. 定位new运算符
 11. 实战项目：队列模拟
 12. 编程练习
-

1. 动态内存分配基础

1.1 为什么需要动态内存分配？

在实际编程中，我们经常遇到这样的问题：

- 不知道需要存储多少数据
- 不同对象需要不同大小的存储空间
- 希望在运行时根据需要分配内存

CPP

// 传统方法的问题

```
class Person {  
private:  
    char name[50]; // 固定大小, 可能浪费或不够用  
public:  
    Person(const char* n) {  
        strcpy(name, n); // 如果n超过49个字符会出错  
    }  
};
```

// 动态内存分配的解决方案

```
class Person {  
private:  
    char* name; // 指针, 可指向任意大小的内存  
    int nameLen;  
public:  
    Person(const char* n) {  
        nameLen = strlen(n);  
        name = new char[nameLen + 1]; // 动态分配恰好需要的内存  
        strcpy(name, n);  
    }  
  
    ~Person() {  
        delete[] name; // 释放内存  
    }  
};
```

1.2 使用new和delete的基本规则

CPP

// 基本用法

```
int* ptr = new int(42); // 分配单个int  
delete ptr; // 释放单个对象
```

```
int* arr = new int[10]; // 分配数组  
delete[] arr; // 释放数组
```

// 重要规则:

// 1. new 对应 delete

// 2. new[] 对应 delete[]

// 3. 每个new都必须有对应的delete

// 4. 不要多次delete同一个指针

2. 特殊成员函数详解

C++编译器会自动生成6个特殊成员函数（在C++11中）：

2.1 自动生成的成员函数

函数类型	何时生成	默认行为
默认构造函数	没有定义任何构造函数时	不执行任何操作
复制构造函数	没有定义时	逐成员复制
赋值运算符	没有定义时	逐成员赋值
析构函数	没有定义时	不执行任何操作
移动构造函数	C++11新增	移动语义
移动赋值运算符	C++11新增	移动语义

2.2 问题示例：StringBad类

CPP

```

// 有缺陷的字符串类
class StringBad {
private:
    char* str;
    int len;
    static int num_strings; // 静态成员

public:
    StringBad(const char* s);
    StringBad();
    ~StringBad();

    friend std::ostream& operator<<(std::ostream& os, const StringBad& st);
};

int StringBad::num_strings = 0; // 静态成员初始化

// 构造函数
StringBad::StringBad(const char* s) {
    len = std::strlen(s);
    str = new char[len + 1];
    std::strcpy(str, s);
    num_strings++;
    std::cout << num_strings << ": \"" << str << "\" object created\n";
}

// 默认构造函数
StringBad::StringBad() {
    len = 4;
    str = new char[4];
    std::strcpy(str, "C++");
    num_strings++;
    std::cout << num_strings << ": \"" << str << "\" default object created\n";
}

// 析构函数
StringBad::~~StringBad() {
    std::cout << "\"" << str << "\" object deleted, ";
    --num_strings;
    std::cout << num_strings << " left\n";
    delete[] str;
}

```

2.3 StringBad的问题

```
void problemDemo() {
    StringBad headline1("Celery Stalks at Midnight");
    StringBad headline2("Lettuce Prey");

    // 问题1: 复制构造函数
    StringBad sports = headline1; // 浅复制!

    // 问题2: 赋值运算符
    StringBad knot;
    knot = headline1; // 浅复制!

    // 结果: 多个对象指向同一块内存
    // 析构时会多次删除同一块内存 -> 程序崩溃!
}
```

3. 复制构造函数

3.1 什么是复制构造函数?

复制构造函数用于将一个对象初始化为同类的另一个对象。

CPP

```
class MyClass {
public:
    MyClass(const MyClass& other); // 复制构造函数
};
```

3.2 何时调用复制构造函数?

CPP

```
MyClass obj1("Hello");
MyClass obj2(obj1);    // 1. 显式初始化
MyClass obj3 = obj1;   // 2. 初始化语法
MyClass obj4 = MyClass(obj1); // 3. 临时对象初始化

void func(MyClass obj) { } // 4. 按值传递参数
MyClass func2() {
    MyClass temp;
    return temp;    // 5. 按值返回对象
}

func(obj1);    // 调用复制构造函数
MyClass result = func2(); // 调用复制构造函数
```

3.3 正确的复制构造函数实现

CPP

```
class String {
private:
    char* str;
    int len;
    static int num_strings;

public:
    // 复制构造函数 - 深度复制
    String(const String& st) {
        num_strings++; // 更新计数
        len = st.len; // 复制长度
        str = new char[len + 1]; // 分配新内存
        std::strcpy(str, st.str); // 复制字符串内容
        std::cout << "复制构造函数被调用\n";
    }

    // 其他成员函数...
};
```

3.4 深度复制vs浅度复制对比

CPP

```
// 浅度复制（默认行为）- 危险！
void shallowCopy() {
    String original("Hello");
    String copy = original; // 两个对象指向同一内存
    // 析构时：删除同一内存两次 -> 崩溃！
}

// 深度复制（正确做法）
void deepCopy() {
    String original("Hello");
    String copy = original; // 创建内存的独立副本
    // 析构时：各自删除自己的内存 -> 安全！
}
```

4. 赋值运算符重载

4.1 赋值运算符的特点

CPP

```
class String {
public:
    String& operator=(const String& st); // 返回引用以支持连续赋值
};
```

4.2 正确的赋值运算符实现

CPP

```
String& String::operator=(const String& st) {
    // 1. 检查自我赋值
    if (this == &st) {
        return *this;
    }

    // 2. 释放旧内存
    delete[] str;

    // 3. 分配新内存并复制数据
    len = st.len;
    str = new char[len + 1];
    std::strcpy(str, st.str);

    // 4. 返回对象引用
    return *this;
}
```

4.3 为什么需要检查自我赋值？

CPP

```
void selfAssignmentProblem() {
    String s1("Hello");
    s1 = s1; // 自我赋值

    // 如果没有检查：
    // 1. delete[] str; <- 删除s1的内存
    // 2. strcpy(str, st.str); <- st.str已经被删除！程序崩溃
}
```

4.4 支持连续赋值

CPP

```
String s1, s2, s3;  
s3 = s2 = s1; // 等价于 s3 = (s2 = s1)  
  
// 这要求operator=返回对象引用  
String& operator=(const String& st) {  
    // ... 实现  
    return *this; // 返回引用支持连续赋值  
}
```

5. 深度复制与浅度复制

5.1 概念对比

C++

```

class DeepCopyDemo {
private:
    char* data;
    int size;

public:
    // 浅度复制 (默认行为)
    void shallowCopyExample() {
        DeepCopyDemo obj1("Hello");
        DeepCopyDemo obj2 = obj1; // 默认复制构造函数

        // 内存布局:
        // obj1.data -> [H|e|l|l|o|\0]
        // obj2.data -> 同一块内存!

        // 问题: 修改一个会影响另一个
        obj1.data[0] = 'X';
        // 现在obj2.data也变成了"Xello"
    }

    // 深度复制 (正确做法)
    DeepCopyDemo(const DeepCopyDemo& other) {
        size = other.size;
        data = new char[size + 1]; // 分配新内存
        std::strcpy(data, other.data); // 复制内容

        // 内存布局:
        // obj1.data -> [H|e|l|l|o|\0]
        // obj2.data -> [H|e|l|l|o|\0] (不同的内存块)
    }
};

```

5.2 何时需要深度复制?

需要深度复制的情况:

- 类包含指向动态分配内存的指针
- 类管理系统资源 (文件句柄、网络连接等)
- 类包含其他需要深度复制的对象

不需要深度复制的情况:

- 类只包含基本数据类型
- 类包含标准库容器 (它们已经实现了深度复制)
- 类设计为不可复制

6. 静态类成员

6.1 静态数据成员

CPP

```
class Counter {
private:
    static int count;    // 静态数据成员声明
    int id;

public:
    Counter() {
        id = ++count;    // 每个对象都有唯一ID
    }

    static int getCount() { // 静态成员函数
        return count;
    }

    void showInfo() const {
        std::cout << "对象ID: " << id << ", 总数: " << count << std::endl;
    }
};

// 静态成员定义 (在类外)
int Counter::count = 0;

void staticMemberDemo() {
    std::cout << "初始计数: " << Counter::getCount() << std::endl;

    Counter obj1; // count = 1
    Counter obj2; // count = 2
    Counter obj3; // count = 3

    obj1.showInfo(); // 对象ID: 1, 总数: 3
    obj2.showInfo(); // 对象ID: 2, 总数: 3
    obj3.showInfo(); // 对象ID: 3, 总数: 3
}
```

6.2 静态成员函数特点

CPP

```

class MyClass {
private:
    static int staticVar;
    int instanceVar;

public:
    static void staticFunction() {
        staticVar = 10;    // ✓ 可以访问静态成员
        // instanceVar = 20; // ✗ 不能访问实例成员
        // this->instanceVar = 20; // ✗ 静态函数没有this指针
    }

    void instanceFunction() {
        staticVar = 30;    // ✓ 实例函数可以访问静态成员
        instanceVar = 40; // ✓ 可以访问实例成员
    }
};

// 调用方式
MyClass::staticFunction(); // 通过类名调用
MyClass obj;
obj.staticFunction();      // 通过对象调用（不推荐）

```

7. 运算符重载进阶

7.1 重载<<运算符（输出）

```

CPP

class String {
    friend std::ostream& operator<<(std::ostream& os, const String& st);
};

std::ostream& operator<<(std::ostream& os, const String& st) {
    os << st.str;
    return os; // 返回引用支持连续输出
}

void outputDemo() {
    String s1("Hello");
    String s2("World");
    std::cout << s1 << " " << s2 << std::endl; // 连续输出
}

```

7.2 重载>>运算符（输入）

CPP

```
class String {
    static const int CINLIM = 80;
    friend std::istream& operator>>(std::istream& is, String& st);
};

std::istream& operator>>(std::istream& is, String& st) {
    char temp[String::CINLIM];
    is.get(temp, String::CINLIM);
    if (is) {
        st = temp; // 使用赋值运算符
    }
    while (is && is.get() != '\n') {
        continue; // 丢弃多余字符
    }
    return is;
}
```

7.3 重载[]运算符（下标访问）

CPP

```
class String {
public:
    // 非const版本 - 可读写
    char& operator[](int i) {
        return str[i];
    }

    // const版本 - 只读
    const char& operator[](int i) const {
        return str[i];
    }
};

void subscriptDemo() {
    String s("Hello");
    const String cs("World");

    s[0] = 'h';    // 调用非const版本, 可修改
    char c = cs[0]; // 调用const版本, 只读
    // cs[0] = 'w'; // 编译错误! const对象不能修改
}
```

7.4 重载比较运算符

CPP

```
class String {  
public:  
    friend bool operator<(const String& st1, const String& st2);  
    friend bool operator>(const String& st1, const String& st2);  
    friend bool operator==(const String& st1, const String& st2);  
};  
  
bool operator<(const String& st1, const String& st2) {  
    return std::strcmp(st1.str, st2.str) < 0;  
}  
  
bool operator>(const String& st1, const String& st2) {  
    return st2 < st1; // 利用已定义的<运算符  
}  
  
bool operator==(const String& st1, const String& st2) {  
    return std::strcmp(st1.str, st2.str) == 0;  
}
```

8. 构造函数中使用new的注意事项

8.1 基本规则

CPP

```

class SafeString {
private:
    char* str;
    int len;

public:
    // ✔️ 正确的构造函数
    SafeString(const char* s) {
        len = std::strlen(s);
        str = new char[len + 1]; // 使用new[]
        std::strcpy(str, s);
    }

    // ✔️ 正确的析构函数
    ~SafeString() {
        delete[] str; // 使用delete[]与new[]配对
    }

    // ❌ 错误示例
    void badExamples() {
        // 错误1: new和delete不配对
        str = new char[10];
        delete str; // 应该是delete[]

        // 错误2: 没有分配足够内存
        str = new char[len]; // 缺少'\0'的空间

        // 错误3: 直接赋值指针
        str = s; // 没有复制内容, 只是共享指针
    }
};

```

8.2 成员初始化列表

CPP

```
class MyClass {
private:
    const int size;    // const成员必须在初始化列表中初始化
    int& ref;          // 引用必须在初始化列表中初始化
    char* data;

public:
    // ✔ 使用成员初始化列表
    MyClass(int s, int& r) : size(s), ref(r) {
        data = new char[size];
    }

    // ✘ 错误: const和引用不能在构造函数体中赋值
    MyClass(int s, int& r) {
        size = s;      // 编译错误!
        ref = r;       // 编译错误!
        data = new char[size];
    }
};
```

8.3 处理多个构造函数

CPP

```
class FlexibleString {
private:
    char* str;
    int len;

public:
    // 构造函数1：从C字符串
    FlexibleString(const char* s) {
        len = std::strlen(s);
        str = new char[len + 1];
        std::strcpy(str, s);
    }

    // 构造函数2：默认构造函数
    FlexibleString() {
        len = 0;
        str = new char[1];    // 而不是str = nullptr
        str[0] = '\0';
    }

    // 析构函数：与所有构造函数兼容
    ~FlexibleString() {
        delete[] str;        // 对所有情况都正确
    }
};
```

9. 指向对象的指针

9.1 基本用法

CPP

```
void pointerDemo() {  
    // 1. 声明指向对象的指针  
    String* ptr;  
  
    // 2. 指向已有对象  
    String s1("Hello");  
    ptr = &s1;  
  
    // 3. 使用new创建对象  
    String* ptr2 = new String("World");  
    String* ptr3 = new String();    // 调用默认构造函数  
  
    // 4. 通过指针访问成员  
    std::cout << ptr->length() << std::endl;    // 使用->  
    std::cout << (*ptr2).length() << std::endl;    // 使用*和  
  
    // 5. 释放动态分配的对象  
    delete ptr2;  
    delete ptr3;  
    // 注意：不要delete ptr，因为它指向栈对象s1  
}
```

9.2 指针数组vs对象数组

CPP

```

void arrayComparison() {
    // 对象数组
    String strings[3] = {
        String("First"),
        String("Second"),
        String("Third")
    };

    // 指针数组 (更灵活)
    String* ptrs[3];
    ptrs[0] = new String("Dynamic First");
    ptrs[1] = new String("Dynamic Second");
    ptrs[2] = new String("Dynamic Third");

    // 使用
    for (int i = 0; i < 3; i++) {
        std::cout << strings[i] << std::endl; // 直接访问
        std::cout << *ptrs[i] << std::endl; // 通过指针访问
    }

    // 清理指针数组
    for (int i = 0; i < 3; i++) {
        delete ptrs[i];
    }
}

```

9.3 智能指针 (C++11)

CPP

```

#include <memory>

void smartPointerDemo() {
    // unique_ptr: 独占所有权
    std::unique_ptr<String> uptr(new String("Unique"));
    // 或者使用make_unique (C++14)
    auto uptr2 = std::make_unique<String>("Better");

    // shared_ptr: 共享所有权
    std::shared_ptr<String> sptr1(new String("Shared"));
    std::shared_ptr<String> sptr2 = sptr1; // 共享同一对象

    // 自动管理内存, 无需手动delete
}

```

10. 定位new运算符

10.1 什么是定位new?

定位new允许你在指定的内存位置构造对象。

CPP

```
#include <new>

void placementNewDemo() {
    // 1. 分配原始内存
    char buffer[1024];

    // 2. 在指定位置构造对象
    String* ptr1 = new(buffer) String("First");
    String* ptr2 = new(buffer + sizeof(String)) String("Second");

    // 3. 使用对象
    std::cout << *ptr1 << std::endl;
    std::cout << *ptr2 << std::endl;

    // 4. 手动调用析构函数
    ptr2->~String();
    ptr1->~String();

    // 5. 不要使用delete，因为内存不是用new分配的
    // delete ptr1; // 错误!
    // delete ptr2; // 错误!
}
```

10.2 定位new的注意事项

CPP

```
class PlacementExample {
private:
    std::string data;

public:
    PlacementExample(const std::string& s) : data(s) {
        std::cout << "构造: " << data << std::endl;
    }

    ~PlacementExample() {
        std::cout << "析构: " << data << std::endl;
    }
};

void placementNewRules() {
    char buffer[512];

    // 1. 确保内存对齐
    PlacementExample* ptr1 = new(buffer) PlacementExample("First");

    // 2. 确保不重叠
    PlacementExample* ptr2 = new(buffer + sizeof(PlacementExample))
        PlacementExample("Second");

    // 3. 按相反顺序销毁
    ptr2->~PlacementExample(); // 后构造的先销毁
    ptr1->~PlacementExample(); // 先构造的后销毁

    // 4. 缓冲区在所有对象销毁后才能释放
}
```

11. 实战项目：队列模拟

11.1 队列类设计

cpp

```
// 队列中的数据项
```

```
class Customer {  
private:  
    long arrive;    // 到达时间  
    int processtime; // 处理时间  
  
public:  
    Customer() : arrive(0), processtime(0) {}  
  
    void set(long when) {  
        processtime = std::rand() % 3 + 1; // 1-3分钟  
        arrive = when;  
    }  
  
    long when() const { return arrive; }  
    int ptime() const { return processtime; }  
};
```

```
typedef Customer Item;
```

```
// 队列类
```

```
class Queue {  
private:  
    struct Node {  
        Item item;  
        struct Node* next;  
    };  
  
    enum { Q_SIZE = 10 };  
    Node* front;    // 队首指针  
    Node* rear;     // 队尾指针  
    int items;      // 当前项目数  
    const int qsize; // 最大容量  
  
    // 禁止复制  
    Queue(const Queue& q) : qsize(0) {}  
    Queue& operator=(const Queue& q) { return *this; }  
  
public:  
    Queue(int qs = Q_SIZE);  
    ~Queue();  
  
    bool isempty() const { return items == 0; }  
    bool isfull() const { return items == qsize; }  
    int queuecount() const { return items; }  
    bool enqueue(const Item& item); // 入队
```

```
bool dequeue(Item& item);    // 出队  
};
```

11.2 队列实现

cpp

```
Queue::Queue(int qs) : qsize(qs) {  
    front = rear = nullptr;  
    items = 0;  
}
```

```
Queue::~~Queue() {  
    Node* temp;  
    while (front != nullptr) {  
        temp = front;  
        front = front->next;  
        delete temp;  
    }  
}
```

```
bool Queue::enqueue(const Item& item) {  
    if (isfull()) {  
        return false;  
    }
```

```
    Node* add = new Node;  
    add->item = item;  
    add->next = nullptr;  
    items++;
```

```
    if (front == nullptr) {    // 队列为空  
        front = add;  
    } else {                  // 队列不为空  
        rear->next = add;  
    }  
    rear = add;  
    return true;  
}
```

```
bool Queue::dequeue(Item& item) {  
    if (front == nullptr) {  
        return false;  
    }
```

```
    item = front->item;  
    items--;  
    Node* temp = front;  
    front = front->next;  
    delete temp;
```

```
    if (items == 0) {  
        rear = nullptr;
```

```
}  
return true;  
}
```

11.3 ATM模拟程序

CPP

```
const int MIN_PER_HR = 60;

bool newcustomer(double x) {
    return (std::rand() * x / RAND_MAX < 1);
}

int main() {
    std::srand(std::time(0));

    std::cout << "ATM队列模拟\n";
    std::cout << "队列最大长度: ";
    int qs;
    std::cin >> qs;
    Queue line(qs);

    std::cout << "模拟小时数: ";
    int hours;
    std::cin >> hours;
    long cyclelimit = MIN_PER_HR * hours;

    std::cout << "每小时平均客户数: ";
    double perhour;
    std::cin >> perhour;
    double min_per_cust = MIN_PER_HR / perhour;

    // 统计变量
    Item temp;
    long turnaways = 0; // 被拒绝的客户
    long customers = 0; // 接受的客户
    long served = 0; // 已服务的客户
    long sum_line = 0; // 累计队列长度
    int wait_time = 0; // ATM忙碌时间
    long line_wait = 0; // 累计等待时间

    // 模拟循环
    for (int cycle = 0; cycle < cyclelimit; cycle++) {
        // 检查是否有新客户
        if (newcustomer(min_per_cust)) {
            if (line.isfull()) {
                turnaways++;
            } else {
                customers++;
                temp.set(cycle);
                line.enqueue(temp);
            }
        }
    }
}
```

```

// 处理队首客户
if (wait_time <= 0 && !line.isempty()) {
    line.dequeue(temp);
    wait_time = temp.ptime();
    line_wait += cycle - temp.when();
    served++;
}

if (wait_time > 0) {
    wait_time--;
}

sum_line += line.queuecount();
}

// 输出结果
if (customers > 0) {
    std::cout << "接受客户: " << customers << std::endl;
    std::cout << "服务客户: " << served << std::endl;
    std::cout << "拒绝客户: " << turnaways << std::endl;
    std::cout << "平均队列长度: ";
    std::cout << std::fixed << std::setprecision(2);
    std::cout << (double) sum_line / cyclelimit << std::endl;
    std::cout << "平均等待时间: "
        << (double) line_wait / served << " 分钟\n";
} else {
    std::cout << "没有客户!\n";
}

std::cout << "模拟结束!\n";
return 0;
}

```

12. 编程练习

练习1：完善的String类

实现一个功能完整的String类：

CPP

```
class String {
private:
    char* str;
    int len;
    static int num_strings;

public:
    // 构造函数
    String(const char* s);
    String();
    String(const String& st);

    // 析构函数
    ~String();

    // 赋值运算符
    String& operator=(const String& st);
    String& operator=(const char* s);

    // 访问运算符
    char& operator[](int i);
    const char& operator[](int i) const;

    // 比较运算符
    friend bool operator<(const String& st1, const String& st2);
    friend bool operator>(const String& st1, const String& st2);
    friend bool operator==(const String& st1, const String& st2);

    // 输入输出运算符
    friend std::ostream& operator<<(std::ostream& os, const String& st);
    friend std::istream& operator>>(std::istream& is, String& st);

    // 成员函数
    int length() const { return len; }
    void stringup();    // 转换为大写
    void stringlow();   // 转换为小写
    int has(char c) const; // 计算字符出现次数

    // 静态函数
    static int HowMany() { return num_strings; }
};

// 实现提示
String& String::operator=(const char* s) {
    delete[] str;
    len = std::strlen(s);
```

```
    str = new char[len + 1];
    std::strcpy(str, s);
    return *this;
}

void String::stringup() {
    for (int i = 0; i < len; i++) {
        str[i] = std::toupper(str[i]);
    }
}

int String::has(char c) const {
    int count = 0;
    for (int i = 0; i < len; i++) {
        if (str[i] == c) {
            count++;
        }
    }
    return count;
}
```

练习2：动态Stack类

cpp

```
class Stack {
private:
    int* items;    // 动态数组
    int size;      // 栈大小
    int top;       // 栈顶索引

public:
    Stack(int n = 10);           // 构造函数
    Stack(const Stack& st);      // 复制构造函数
    ~Stack();                    // 析构函数

    bool isempty() const;
    bool isfull() const;
    bool push(int item);        // 入栈
    bool pop(int& item);        // 出栈

    Stack& operator=(const Stack& st); // 赋值运算符
};

// 实现
Stack::Stack(int n) : size(n), top(0) {
    items = new int[size];
}

Stack::Stack(const Stack& st) : size(st.size), top(st.top) {
    items = new int[size];
    for (int i = 0; i < top; i++) {
        items[i] = st.items[i];
    }
}

Stack::~~Stack() {
    delete[] items;
}

Stack& Stack::operator=(const Stack& st) {
    if (this == &st) {
        return *this;
    }

    delete[] items;
    size = st.size;
    top = st.top;
    items = new int[size];

    for (int i = 0; i < top; i++) {
```

```
        items[i] = st.items[i];  
    }  
  
    return *this;  
}
```

练习3：智能指针模拟

CPP

```
template<typename T>
class SmartPtr {
private:
    T* ptr;
    int* ref_count; // 引用计数

public:
    explicit SmartPtr(T* p = nullptr) : ptr(p) {
        if (ptr) {
            ref_count = new int(1);
        } else {
            ref_count = nullptr;
        }
    }

    SmartPtr(const SmartPtr& sp) : ptr(sp.ptr), ref_count(sp.ref_count) {
        if (ref_count) {
            (*ref_count)++;
        }
    }

    ~SmartPtr() {
        if (ref_count) {
            (*ref_count)--;
            if (*ref_count == 0) {
                delete ptr;
                delete ref_count;
            }
        }
    }

    SmartPtr& operator=(const SmartPtr& sp) {
        if (this != &sp) {
            // 减少当前对象的引用计数
            if (ref_count) {
                (*ref_count)--;
                if (*ref_count == 0) {
                    delete ptr;
                    delete ref_count;
                }
            }

            // 复制新的指针和引用计数
            ptr = sp.ptr;
            ref_count = sp.ref_count;
            if (ref_count) {
                (*ref_count)++;
            }
        }
    }
};
```

```

        (*ref_count)++;
    }
}
return *this;
}

T& operator*() { return *ptr; }
T* operator->() { return ptr; }
T* get() const { return ptr; }
int use_count() const { return ref_count ? *ref_count : 0; }
};

// 使用示例
void smartPtrDemo() {
    SmartPtr<String> sp1(new String("Hello"));
    SmartPtr<String> sp2 = sp1; // 引用计数变为2

    std::cout << "引用计数: " << sp1.use_count() << std::endl; // 输出: 2
    std::cout << *sp1 << std::endl; // 输出: Hello

    {
        SmartPtr<String> sp3 = sp1; // 引用计数变为3
        std::cout << "引用计数: " << sp1.use_count() << std::endl; // 输出: 3
    } // sp3析构, 引用计数变为2

    std::cout << "引用计数: " << sp1.use_count() << std::endl; // 输出: 2
} // sp1和sp2析构, String对象被自动删除

```

💡 重要知识点总结

动态内存管理的黄金法则

1. 配对原则

- new 配 delete
- new[] 配 delete[]
- 每个都要有对应的 new delete

2. 深度复制三法则

- 如果需要自定义析构函数, 通常也需要自定义复制构造函数和赋值运算符
- 如果需要自定义复制构造函数, 通常也需要自定义析构函数和赋值运算符
- 如果需要自定义赋值运算符, 通常也需要自定义析构函数和复制构造函数

3. C++11的五法则

- 在三法则基础上, 还需要考虑移动构造函数和移动赋值运算符

常见错误及避免方法

错误类型	问题描述	解决方案
内存泄漏	忘记delete	在析构函数中delete，使用智能指针
重复删除	多次delete同一指针	实现深度复制，delete后置nullptr
悬空指针	使用已删除的指针	避免浅复制，使用引用计数
数组越界	访问超出范围的内存	边界检查，使用容器类

设计建议

1. 优先使用标准库

- `std::string` 代替 `char*`
- `std::vector` 代替 动态数组
- `std::shared_ptr` 代替 原始指针

2. RAII原则

- 资源获取即初始化
- 在构造函数中获取资源
- 在析构函数中释放资源

3. const正确性

- 尽可能使用const
- 提供const和非const版本的成员函数

通过本手册的学习，你应该能够：

- 理解动态内存分配的原理和用法
- 正确实现包含指针成员类
- 掌握复制构造函数和赋值运算符的设计
- 避免常见的内存管理错误
- 设计出安全、高效的C++类

继续练习和实践，你将能够熟练运用这些技能来解决实际的编程问题！